Approach

Search Algorithm

I used a **depth-limited Minimax search with Alpha-beta pruning**, chosen primarily due to its efficiency in adversarial search environments with limited search depth, a requirement brought on by the 250MB space limitation and 180s computation limit. Minimax provides optimal moves by evaluating game states to a dynamically changing depth. While Alpha-Beta pruning significantly reduces the number of explored nodes to enhance computational efficiency.

Notably a few implementation decisions were made in addition to the traditional Minimax base algorithm:

- Iterative deepening is combined with a strict time limit (SEARCH_TIME_LIMIT) to ensure a robust yet timely response. This number was determined by an analysis of simulated games: In over 200 simulated games against numerous benchmarking agents (Random, Greedy, Adversarial Search): The longest game length was 72 moves, with the average hovering around 56.87, thus a 5s response time will still fit under the 180s total computation limit.
- **End game VS Early/Mid Game Adjustments**
    - In the early game/mid game, I searched to a base depth of 3 plies, whereas in the **end game** (defined as ≥ 4 frogs' home) we dynamically extend to 4 plies so that our shrinking move set still allows deeper look-ahead under the same time budget. An even ply depth here was a suitable choice as we should be solving for our optimal move during the end game and getting to our target the soonest.
    - In addition to this, a shortest path search algorithm is implemented using a BFS after 5 frogs have reached the end, to identify the shortest guaranteed finish sequence for the final frog, bypassing Minimax entirely in this case to save computation time and avoid redundant grow actions.
- An "immediate-win" check is added to simulate legal moves and immediately return any moves that places all six frogs on the goal row, to eliminate chances of exploring inferior grow actions.
- An **odd ply depth** (3 plies) was selected specifically due to the observed strategic advantage and computational balance comparative to the same parameters and heuristic functions tested on even ply depths (2 or 4). This suggests that an odd ply depths tend to provide better tactical foresight by ending the search at the opponent's move [reflects our state after their turn], enabling more accurate evaluation of threats and opportunities within the tight time constraints.
    - 3 was selected over 5 due to efficiency constraints due to the high branching factor of actions, especially in the mid game. A Zobrist-keyed transposition table was investigated to potentially increase the ply depth to 5 (not ultimately implemented, due to: low likelihood of transpositions occurring more than once at a shallow depth of 3-4 plies, high volatility of Freckers states and memory constraints)

Evaluation Function

Heuristics

I tested a breadth of evaluation functions, starting with the sum of Manhattan Distance from the frogs to the goal row and slowly building more complex heuristics, such as jump potential, mobility,

concentration bonuses and more. This was done in an iterative approach, testing each agent iteratively against each other and other benchmark agents to evaluate the most optimal solution.

The **final agent** was composed of the following heuristics in the evaluation function:

- **Goal Saturation (weighted highly):** Rewards having frogs on the goal row, strongly aligning agent strategy towards completing victory conditions of Freckers of having all 6 frogs in the goal row.
- **Chebyshev Distance to Goal:** Evaluates relative positional advantage by computing the cumulative distances that frogs still need to reach their respective goals. Minimising the delta of this metric (between agent and opponent) encourages steady forward progression whilst blocking high potential moves for the opponent. It also rewards moves such as jumps that are deemed to be more efficient. Other potential distance metrics were also considered: including Manhattan and Euclidean, however Chebyshev was deemed more akin to the game of Freckers, due to the potential of diagonal movements.
- **Mobility:** Assesses the availability of moves, incentivizing positions offering greater future flexibility, which by extension gives us a higher potential for promising moves, which is important due to the low ply-depth of 3. This is also inspired by the mobility heuristic commonly used in games such as Chess, to preserve options and facilitate adaptive gameplay to dynamic opponent strategies.
- **Threat Detection:** Penalizes board positions that expose frogs to enemy jumps, thus strategically promoting defensive positioning and risk minimization. This concept is analogous to threat detection heuristics used in chess, where maintaining defensive integrity is critical to strategic resilience (Campbell et al., 2002).
- **Concentration Bonus:** Encourages clustering of frogs by evaluating the inverse average pairwise distances between them. This heuristic is inspired by research conducted in similar games such as Chinese checkers, whereby, proximity increases potential for coordinated advancements whilst also providing defensive strength.
- **Growth Potential:** Rewards positions with potential lily pad growth, effectively managing the spatial expansion and long-term strategic control. This aligns with territory control heuristics found in games like Go, emphasizing foresight in spatial expansion (Bouzy & Cazenave, 2001)
- **Jump Potential:** Encourages positions with higher jump potentials enhances offensive capabilities while simultaneously maintaining vigilance against opponents' threats, this is quite similar to Threat detection, and similar heuristics are observed in agents for strategic board games with jumping mechanics such as Checkers (Schaeffer et al., 2007).

<u>Weights and other considerations</u>

The relative weights were also determined through benchmarking against both baseline, greedy and adversarial search agents, with an iteration process to determine the ultimate weighting. (I.e Goal Saturation was weighted 1000.0 to incentivise moves that drive towards endgame, jump and mobility are weighted modestly to reflect their high frequency and diminishing marginal value per additional action) Other heuristics were also considered, however, did not ultimately show improved efficiency or were too intensive, these include but are not limited to:

- **Euclidean Distance to Lilypad:** Similar to Mobility, but doesn't directly give us useful information, and potentially adding noise to jump potential and mobility.
- **Penalties for frogs blocking together and creating bottlenecks:** Similar to above, this is mostly already represented for in mobility.

- **Conduct fast greedy pathfinder (A*) for each frog:** Determined to be too computationally intensive and may compromise the quality of other heuristics. A similar implementation was however used for end-game valuation.

Performance Evaluation:

Methodology:

My agent was rigorously tested against multiple benchmark opponents, aligned with the project specs:

- **Random Move Agents (Baseline)**
- **Greedy Agents** focused on immediate board advantages without deeper strategic foresight – These were tested using a one ply adversarial search agent, along each step of the developmental process, and our final agent was also benchmarked against each.
- **Adversarial search agent** with numerous iterations of Minimax search with different heuristics, weightings and strategies

These were tested specifically for win-rate, average time thought and average total number of moves required for winning. With baseline and greedy agents tested for 50 runs, whereas the adversarial search agents were compared with each other 10 times each. The colours rotate after each run.

Results [Win rates]:

Model Version	Random	Greedy	Adversarial
V21 (Final)	96%	96%	60% against V8 100% against V14 100% against V17
V8 (Manhattan + Jumps)	94%	94%	**N/A**
V14 (Similar to V21-tweaked weights, no jumps)	96%	94%	**N/A**
V17 (Most Similar to V21, no jumps)	96%	96%	**N/A**

V21 Specifically:

- **Against Random Agents:** 96%-win rate, no losses, draws caused by repeated grow actions rather than finishing the game. Average Think Time of 1.56s, Average Turns to Win: 57.49
- **Against Greedy Agent** (1-ply deep search focussed on maximising Manhattan distance [V8 without jump potential heuristic])**:** 96%-win rate, no losses, draws caused by repeated grow actions rather than finishing the game. Average Think Time of 1.42s, Average Turns to Win: 53.94
- **Against Adversarial Search Agent (10 Games played Against V8, V14, V19)**
    - **V8:** 60% Win rate, Avg Moves: 54.9, Average Think of 1.25s
    - **V14:** 100% Win rate, Avg Moves 57.5, Average Think of 1.56s
    - **V17:** 100% Win rate, Avg Moves 57.3, Average Think of 1.51s

Comparative Analysis:

My iterative testing and refinement suggested a slight advantage of our current multi-dimensional heuristic approach that optimises for both aggressive and defensive capabilities, whereby adjustments to heuristic weighting were determined through systematic experimentation.

The other strong contenders of my final agent were V8, and V14 and V17. V8 had the simplest and most efficient evaluation metric, of having a sum of the differences of the Manhattan distances to the

goal row, weighted 5.0 and jump potentials (total number of jumps possible) weighted at 0.1. Although this was faster at decision making due to its simple evaluation heuristic and yielded very similar results against Random Agents and Greedy Agents in terms of win rate, it was slightly weaker against the V21 Adversarial search agent developed. Moreover, time constraints weren't a limiting factor in this scenario, I opted for the V21 with a slightly higher win rate. V14 and V17 were previous iterations of the V21, with slightly adjusted weightings and calculations (Manhattan instead of Chebyshev, lack of jump potential), and both of these proved to be less effective against V21.

Complexity Analysis:

With each player controlling 6 frogs on a fixed 8x8 board for a typical search depth of 3-4 plies, we can conduct an approximate complexity assessment of branching factors, node expansion, and memory usage.

Time Complexity

Let b be the average branching factor (legal moves per state), and d be the search depth (ply), and n be the number of frogs.

- **Branching Factor:** Each of the 6 frogs can attempt several directions (typically 2–5), and more importantly, can chain multiple jump sequences. Since generate_jumps() explores jump paths recursively, branching can also grow in jump-heavy positions. Generally, each frog contributes an average of 2-4 simple moves and 2-3 unique jump sequences, leading to a total branching factor of b ~ 24-42 during midgame.
- **Search Depth:** The agent dynamically adjusts depth to 3 or 4 based on how many frogs have reached the goal row, however it is important to note that 4 ply only comes into use after 4 frogs have reached the goal row, at which point, numerous frogs are no longer considered in the decision making process, and as such the branching factor becomes significantly smaller as it is already during the end game. Thus, the worst-case node expansion will expand into:
    - **d=3:** $O(b^3)$ = 13,824-74,088 nodes
- **Alpha-Beta Pruning:** Should theoretically reduce effective nodes to $O(b^{d/2})$ in the best-case, yielding an estimate of: ~118-272 nodes (d=3)
- **Evaluation Function Cost** (fixed n = 6 frogs)
    - **Chebyshev distance:** O(6 × BOARD_SIZE) = O(1)
    - **Jump potential:** O(6 × j), where j is number of reachable jumps per frog - though the theoretical value for this is high, they upper limit is highly unlikely in practice due to: board boundaries, overlapping paths, occupied landings, lack of reachable lilypads etc.
    - **Threat Detection:** O(6 × j), as it checks for number of reasonable jumps for enemies
    - **Clustering:** O(15) = pairwise distances
    - **Grow Potential:** O(6 x 8) = O(48) – 8 is used due to the number of directions
- **Overall Time:** Worst-case time complexity per action is $O(b^d \times (n^2 + b))$, though pruning and early cutoffs typically reduce actual nodes explored by a few thousand per move, keeping decision time under 5 seconds.

Space Complexity

Through implementing a shallow-copy strategy, I can avoid large recursive memory structures:

- **Board State:** Represented by a dictionary over 64 cells and shallow copied per node → O(1)
- **Frog Sets:** Constant size (6 per player) → O(1)

- **Jump and Threat Structures:** Built on fixed frog count → O(1)
- **Recursion Depth:** Depth-limited stack for Minimax → O(d)

This yields a total space complexity of O(1) according to input size and O(d) stack depth, both of which are small and confine within the 250MB per-agent limit.

Other Aspects:

<u>Technical Innovations and Optimizations</u>

- **Efficient Board Representation:** Designed a LightweightBoard class, reducing overhead by using compact data structures (dict and set) to manage board states and frog positions, minimizing memory usage and facilitating rapid cloning.
- **Optimized Action Generation:** Implemented specialized generate_jumps function, distinctly handling jump actions to reduce redundancy and computation in legal move generation.
    - Moreover, the sequence of move generation is deliberate so that jump moves are done first, followed by moves followed by grow moves, this is so we can try search for most optimal moves in terms of achieving progress towards goal row first, and subsequently likely lead to a higher potential of pruning.
- **Dynamic Time Management:** Employed a responsive depth control mechanism, dynamically adjusting search depth based on game state, performance and available computation time, ensuring optimal moves within constraints.

<u>Additional Research</u>

A thorough dive was conducted into similar game-playing agents, as well as solved games such as Chess, Go, these aided significantly in the development of heuristics [As mentioned before]. Interestingly, the game which I deemed to be most strategically similar to Freckers - Chinese Checkers, was underrepresented in academic literature, despite its rich complexity in multi-jump movement and board traversal.

<u>Alternate Approaches</u>

One alternative approach considered was a **hybrid structure combining Monte Carlo Tree Search (MCTS) with shallow Minimax**, inspired by (Baier & Winands, 2013), who applied this hybrid to effectively improve decision quality in real-time settings with limited search depth. The rationale was that MCTS could rapidly sample longer-term outcomes while a shallow Minimax search could provide tactical stability near the root. However, this approach was ultimately set aside in favor of a pure Minimax strategy with Alpha-Beta pruning due to MCTS's reliance on rollout policies and evaluation consistency, which proved difficult to calibrate given Freckers' small board size, shallow tactical traps, and rapidly shifting lily pad dynamics, ultimately making it hard to converge to the performance level of Minimax.

I also explored advanced action pruning heuristics similar to ProbCut (Buro, 1995) and History Heuristic ordering (Schaeffer, 1989), with the aim of further accelerating Alpha-Beta search through probabilistic cutoffs and dynamic move ordering. These methods were initially prototyped but yielded inconsistent improvements due to the relatively small search space and high variance of jump paths. Instead, a more deterministic move ordering strategy: prioritizing jumps and forward moves was used, which improved pruning effectiveness while maintaining interpretability.

Supporting Work:

Additional Testing and Visualisation

A custom python script (benchmark_freckers.py) was developed to automate extensive matchup simulations, enabling easy synthesis of game statistics and measurement of heuristic efficacy. These include games played, winners, average time thought, and more.

Testing Logs:

Various testing logs have been included in the zip file, these include initial testing logs from V6 up to V21 (Which was the final version used) as well as select agent on agent tests to determine the most efficient agent.

References

- Baier, H., & Winands, M. (2013). Monte-Carlo Tree Search and minimax hybrids, *IEEE Conference on Computational Inteligence in Games (CIG)*, pp. 1-8, doi: 10.1109/CIG.2013.6633630.
- Bouzy, B., & Cazenave, T. (2001). Computer Go: An AI-oriented survey. *Artificial Intelligence*, 132(1), 39-103.
- Buro, M. (1995). ProbCut: An effective selective extension of the alpha-beta algorithm. *ICCGA*, 1(1), 71–76.
- Campbell, M., Hoane Jr., A. J., & Hsu, F. H. (2002). Deep Blue. Artificial Intelligence, 134(1-2), 57-83.
- Schaeffer, J., Burch, N., Björnsson, Y., Kishimoto, A., Müller, M., Lake, R., Lu, P., & Sutphen, S. (2007). Checkers is solved. *Science (New York, N.Y.)*, *317*(5844), 1518–1522. https://doi.org/10.1126/science.1144079
- Wang, Q. *et al.* (2015). Move generation and search strategy research for computer game of Checkers, *The 27th Chinese Control and Decision Conference (2015 CCDC)*, pp. 3727–3731. doi:10.1109/ccdc.2015.7162574.